

Data cleaning process and organization of data with SQL

Session 1

Agenda

- **Data Integrity Types and Risks**

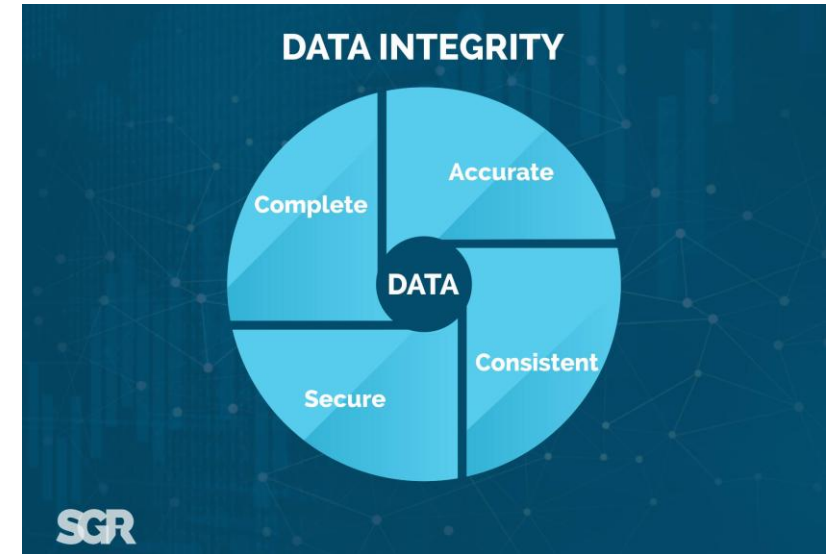
Data Integrity Types and Risks

Importance of Data Integrity in Decision Making: Data integrity is **essential** because it ensures that the data used in decision-making processes is **accurate, reliable, and consistent**. Without data integrity, decisions could be based on **faulty data**, leading to **poor outcomes** and potentially **costly mistakes**.

Overview of SQL for Data Exploration and Cleaning: SQL plays a **crucial role** in ensuring data integrity. It allows users to **retrieve and manipulate data efficiently** and enforce rules (**constraints**) that maintain data integrity in a **relational database**. By using SQL, businesses can ensure their data is **clean, accurate, and ready for analysis**, leading to **better decision-making**.

Data Integrity Importance

- **Foundation for Reliable Decision Making:** Data integrity forms the **foundation for reliable decision-making** by ensuring that the data used is **trustworthy, accurate, and reflective** of real-world situations. For example, in **healthcare**, data integrity ensures that **patient information** is **consistent and accurate** across all systems, preventing potential errors in treatment.
- **Maintains Trust and Credibility:** Maintaining data integrity is **vital** for upholding the **trust of stakeholders**. For instance, **financial institutions** rely on accurate data to manage **customer accounts and transactions**. Any **breach** of data integrity could lead to **loss of trust and credibility**, as well as **legal and financial repercussions**.



Common Types of Integrity Constraints

- **Entity Integrity:** Ensures each record in a database table is **unique and identifiable**, typically through the use of **primary keys**. For example, in a university database, the **student ID** must be **unique** to prevent one student's data from being confused with another's.
- **Referential Integrity:** Maintains **consistency between related tables**. For example, in an e-commerce database, a **foreign key** in the orders table must reference a **valid product ID** from the products table, ensuring that every order corresponds to an **existing product**.
- **Domain Integrity:** Enforces **valid data types, formats, and ranges** for a given column. For instance, a **date field** in an HR database must contain **valid dates**, preventing erroneous or inconsistent entries.

Entity Integrity

- Example:** If we have a customer database and customer_table is present there with attributes like age and name. Then each customer should be uniquely identified. There might be two customers with the same name and same age, so there might be confusion while retrieving the data. If we retrieve the data of customer named 'Angel' then two rows are having this name and there would be confusion. So, to resolve this issues primary keys are assigned in each table and it uniquely identifies each entry of the table.

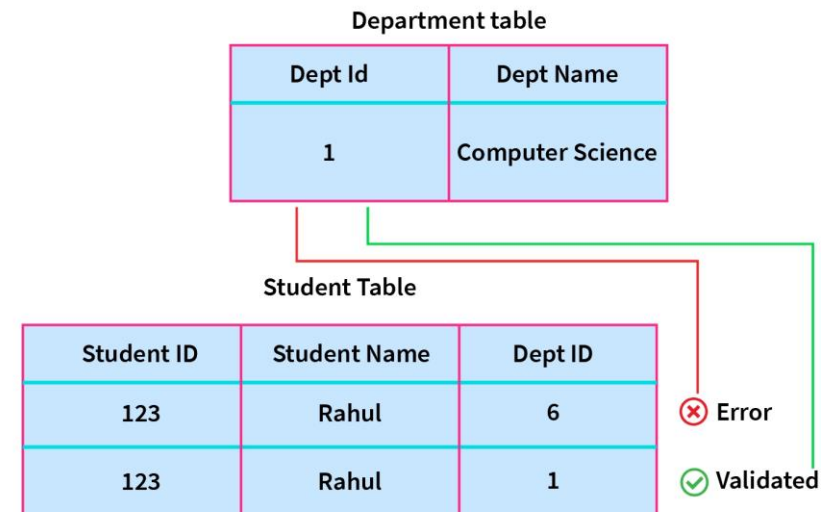
Primary Key	ID	Customer_Name	Age
	1	Andrew	18
	2	Angel	20
		Angel	20

This value cannot be NULL as we will not be able to identify customers uniquely

ENTITY INTEGRITY

Referential Integrity

- **Referential Integrity:** Maintains **consistency between related tables**. For example, in an e-commerce database, a **foreign key** in the orders table must reference a **valid product ID** from the products table, ensuring that every order corresponds to an **existing product**.



Domain Integrity

- Example:** If we have to store the salary of the employees in the ' *employee_table* ' then we can put constraints that it should only be an INTEGER. Any entry other than integer like characters would not be acceptable and when we try to give input like this, the DBMS will produce errors.

Employee_id	Name	Salary	Age
1	Andrew	486522	25
2	Angel	978978	30
3	Anamika	697abc	35

This value is out of domain(not INTEGER)so it is not acceptable.

DOMAIN INTEGRITY

Risks Associated with Data Integrity Issues

Data Corruption:

Example: In an e-commerce system, if customer account data becomes corrupted due to a software bug, users might find their account balances incorrect or their order histories missing. This can lead to frustration and loss of trust in the platform.

Loss of Data:

Example: If a small business accidentally deletes its customer database, it might lose essential information like contact details and purchase history. This can hinder the ability to communicate with customers and impact sales strategies.

Impacts on Decision-Making:

Example: A local restaurant relies on customer feedback data to make menu changes. If the feedback data is incomplete or inaccurate, the restaurant might decide to remove popular dishes, leading to customer dissatisfaction and decreased sales.

Data Integrity Types and Risks

Example 1: Data Corruption Due to Hardware Failure

Scenario: A company's server experiences a **hardware failure**, resulting in **data corruption** within its database.

Consequences:

Inaccurate Reports: In a real-world scenario, a **retail chain** experienced hardware failure during a **peak sales season**, corrupting the sales database. This resulted in **inaccurate sales reports**, leading to **incorrect ordering decisions** and significant **revenue loss**.

Misguided Decisions: In the **financial industry**, a major **bank** experienced data corruption due to a hardware malfunction, leading to **inaccurate account balances**. This confused customers and led to **hasty decisions** by the bank's management to **compensate affected customers**, which later proved unnecessary and costly.

Data Integrity Types and Risks

Example 2: Referential Integrity Violation in Database

Scenario: A database contains **foreign key constraints** linking records between different tables. Due to a **software glitch** or **human error**, a referential integrity constraint is **violated**.

Consequences:

Inconsistent Data: In the **healthcare sector**, a database managing **patient records** and **treatment plans** experienced a referential integrity violation due to a software update. This resulted in **treatment plans** not being correctly linked to the appropriate **patient records**, leading to **delays** and potential **errors in treatment**.

Failed Transactions: In an **e-commerce platform**, a violation of referential integrity due to an **error in a database update** caused transactions to **fail**, leading to a **loss of sales** during a crucial promotional period. This not only affected **revenue** but also damaged the **brand's reputation** among customers.

Entity Integrity: Ensuring Uniqueness and Identifiability

Entity Integrity:

Entity Integrity ensures that each record in a table is unique and can be identified distinctly, typically enforced through primary keys.

Example 1 (Database Context):

In a student database, the **StudentID** serves as the primary key, ensuring that no two students share the same identifier.

Example 2 (Real-World Context):

In a national voter database, a unique identifier such as a Voter ID ensures that no citizen can register more than once or vote multiple times, preventing voter fraud.

Real-World Impact:

Entity integrity is essential in systems like financial databases, where it ensures that each transaction is distinct. Violations could result in duplicate records, leading to issues like double-billing or inaccurate balances.

Entity Integrity: Ensuring Uniqueness and Identifiability

```
CREATE TABLE Students (  
  StudentID INT PRIMARY KEY,  
  FirstName VARCHAR(50),  
  LastName VARCHAR(50)  
);
```

Referential Integrity: Maintaining Consistency Across Tables

Referential Integrity ensures that relationships between tables remain **consistent**. For example, a **foreign key** in one table must correspond to a **primary key** in another.

Example: In an **online bookstore**, the **OrderDetails** table references the **Books** table through the **BookID**. Every **BookID** in **OrderDetails** must exist in **Books**.

Real-World Impact: In a **healthcare system**, ensuring referential integrity prevents **mismatches** between **patient records** and their **treatment histories**, avoiding potential **medical errors**.

Importance: Breaking referential integrity can lead to **orphaned records**, which might result in **incomplete data analysis** and **decision-making errors**.

Referential Integrity: Maintaining Consistency Across Tables

```
CREATE TABLE Orders (  
    OrderID INT PRIMARY KEY,  
    CustomerID INT,  
    OrderDate DATE  
);  
  
CREATE TABLE OrderDetails (  
    OrderDetailID INT PRIMARY KEY,  
    OrderID INT,  
    ProductID INT,  
    Quantity INT,  
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID)  
);
```

Domain Integrity: Ensuring Valid Data Entry

Domain Integrity restricts the values that can be placed in a column based on **data type, format, or range**.

Example: A **salary** column in an **employee table** must be a **positive number**. A **date of birth** field should accept only **valid dates**.

Real-World Impact: In **financial systems**, enforcing domain integrity ensures that all **monetary values** are valid, preventing issues like **negative balances** or **invalid transactions**.

Importance: Domain integrity prevents the entry of **inappropriate data**, which could lead to **incorrect calculations, reports, and decisions**.

Domain Integrity: Ensuring Valid Data Entry

```
CREATE TABLE Employees (  
    EmployeeID INT PRIMARY KEY,  
    FirstName VARCHAR(50),  
    LastName VARCHAR(50),  
    Salary DECIMAL(10, 2) CHECK (Salary > 0),  
    DateOfBirth DATE CHECK (DateOfBirth <= GETDATE())  
);
```

Case Study: Data Integrity Failure in a Banking System

Background: A major **bank** experienced a **data integrity failure** when an update to their **database system** inadvertently allowed **duplicate entries** in the **transaction records**.

Impact:

- **Duplicate Transactions:** Customers were **charged twice** for the same transaction, leading to significant **financial discrepancies**.
- **Loss of Trust:** The bank had to **issue refunds** and deal with **customer complaints**, resulting in a **loss of trust** and a **damaged reputation**.

Conclusions: Ensuring strict **entity** and **referential integrity** could have prevented this issue by automatically **rejecting duplicate transactions** and maintaining **consistent records**.

Case Study: Referential Integrity Violation in E-commerce

Background: An e-commerce platform faced a **referential integrity violation** due to a **software glitch** that deleted **product records** still referenced by **active orders**.

Impact:

- **Failed Transactions:** Customers were **unable to complete purchases** because the products in their carts no longer existed in the **database**.
- **Revenue Loss:** The platform lost **sales** during a critical **promotional period**, impacting its **quarterly earnings**.

Conclusions: Implementing **robust integrity checks** and **thorough testing** before deploying **software updates** can prevent such issues.

Strategies for Mitigating Data Integrity Risks

Regular Audits:

Perform regular data audits to detect and correct integrity issues before they escalate.

Backup and Recovery Plans:

Implement robust backup and recovery procedures to protect against data loss and corruption.

Validation Rules:

Use validation rules and constraints in SQL to enforce data integrity at the point of entry.

Training and Awareness:

Educate employees and developers on the importance of data integrity and best practices to maintain it.

Real-Time Monitoring:

Employ real-time monitoring tools to detect and respond to integrity violations as they occur.

Industry-Specific Applications of Data Integrity

- **Healthcare:** Ensuring **patient records** are **accurate** and **consistent** across different departments and systems to avoid **medical errors**.
- **Finance:** Maintaining **transaction integrity** to prevent **fraud** and ensure **accurate financial reporting**.
- **Retail:** Managing **inventory data integrity** to optimize **stock levels** and prevent loss of sales due to **out-of-stock items**.
- **Government:** Ensuring **voter registration data integrity** to prevent **electoral fraud** and maintain **trust** in the **democratic process**.

Best Practices for Ensuring Data Integrity

- **Enforce Constraints:** Use **SQL constraints** such as **PRIMARY KEY**, **FOREIGN KEY**, and **CHECK** to enforce integrity rules.
- **Implement Transaction Management:** Use **ACID-compliant transactions** to ensure that all database changes are **consistent** and **recoverable**.
- **Regularly Review and Update:** Continuously **review and update** data integrity **policies** and **procedures** to adapt to new **threats** and **challenges**.

Best Practices for Ensuring Data Integrity

- **Enforce Constraints:** Use **SQL constraints** such as **PRIMARY KEY**, **FOREIGN KEY**, and **CHECK** to enforce integrity rules.
- **Implement Transaction Management:** Use **ACID-compliant transactions** to ensure that all database changes are **consistent** and **recoverable**.
- **Regularly Review and Update:** Continuously **review and update** data integrity **policies** and **procedures** to adapt to new **threats** and **challenges**.

Any Questions ?

Thank You